

MomAI

Um Sistema Cognitivo Local com Arquitetura Orientada a Eventos

Whitepaper Tecnico

Autor: github.com/WesleyQDev

Versao: Maio de 2026

Open Source | MIT License

Este documento descreve a arquitetura do MomAI, um assistente virtual local-first que combina modelos de linguagem de grande escala (LLMs) com acoes reais no computador do usuario. Apresenta-se o estado atual do sistema, sua arquitetura em tres componentes principais (Desktop, Node Core, Python Sidecar), o ecossistema de skills e extensoes, e a nova arquitetura MomAI Events para operacao autonoma orientada a eventos. O sistema opera 100% local, sem dependencia de nuvem, utilizando llama.cpp com aceleracao Vulkan para inferencia e LanceDB para memoria vetorial.

Sumario

1 Introducao

2 Arquitetura Atual

- 2.1 Desktop App (Electron + React + TypeScript)
- 2.2 Node Core (Node.js)
- 2.3 Python Sidecar (FastAPI)
- 2.4 Comunicacao entre Componentes

3 Ecossistema de Skills e Extensoes

- 3.1 Skills Core
- 3.2 Extensoes da Comunidade
- 3.3 Skill Discovery

4 Pipeline de Voz

5 MomAI Events: Nova Arquitetura

- 5.1 Tres Pilares Fundamentais
- 5.2 Arquitetura em Tres Camadas
- 5.3 Modulos e Manifestos
- 5.4 O Ciclo Cognitivo em 6 Etapas
- 5.5 Politicas Ativas
- 5.6 Triggers e Auto-Agendamento
- 5.7 Auto-Modificacao de Modulos

6 Integracao Hibrida

7 Roteiro de Desenvolvimento

8 Conclusao

1. Introducao

Assistentes virtuais tradicionais operam como caixas-pretas na nuvem: o usuario envia um comando, o dado viaja para servidores remotos, onde e processado por modelos proprietarios, e a resposta retorna para o dispositivo. Esse modelo apresenta limitacoes fundamentais: dependencia de conexao de internet, custos recorrentes por uso, latencia elevada (500-2000ms), e -- mais critico -- ausencia de privacidade, ja que os dados do usuario sao processados e frequentemente armazenados por terceiros.

A MomAI propoe uma alternativa radical: todo o processamento -- inferencia do modelo de linguagem, armazenamento de memoria, reconhecimento de voz, sintese de fala, e execucao de acoes -- acontece exclusivamente na maquina do usuario. O sistema utiliza llama.cpp com aceleracao Vulkan para inferencia local de LLMs, LanceDB para memoria vetorial, e uma arquitetura de modulos intermediarios que isolam o LLM do hardware fisico.

Este documento esta organizado em duas grandes secoes. As secoes 2 a 4 descrevem a arquitetura atual da MomAI, ja implementada e funcional. A secao 5 apresenta a nova arquitetura MomAI Events, atualmente em fase de projeto, que estende o sistema para operacao autonoma orientada a eventos. A secao 6 discute a integracao entre os dois paradigmas.

2. Arquitetura Atual

A MomAI atual e composta por tres processos independentes que se comunicam via HTTP, WebSocket e IPC (Inter-Process Communication). Cada processo tem responsabilidades bem definidas e pode ser atualizado ou substituido sem impacto sobre os demais.

2.1 Desktop App (Electron + React + TypeScript)

A interface grafica do usuario e construida com Electron 39, React 19 e TypeScript 5.9, utilizando TailwindCSS 3 para estilizacao. O frontend se comunica com o Node Core via HTTP REST para operacoes de chat (streaming SSE), WebSocket para eventos em tempo real, e IPC para comunicacao direta com o processo principal do Electron.

Funcionalidades da interface: (a) chat com streaming de tokens, respostas estruturadas e renderizacao de componentes visuais; (b) modo voz com visualizacao em tempo real do audio; (c) loja de extensoes para instalacao de modulos da comunidade; (d) central de configuracoes; (e) visualizacao de estado do sistema, uso de recursos e historico.

2.2 Node Core (Node.js)

O Node Core e o orquestrador principal do sistema. Escrito em Node.js, ele gerencia todo o fluxo de processamento: desde o recebimento da mensagem do usuario ate a execucao de skills e o streaming da resposta. Seus principais subsistemas sao descritos a seguir.

Gerenciamento de LLM (llama-manager.js)

Gerencia o ciclo de vida completo do llama-server: inicializacao do processo, selecao do backend de GPU (Vulkan, CUDA ou CPU), download automatico de modelos GGUF do HuggingFace, e fallback entre portas em caso de conflito. O sistema suporta tres tiers de modelo: Lite (Qwen3.5-0.8B, contexto 8192), Pro (Qwen3.5-2B, contexto 8192) e Ultra (Qwen3.5-4B, contexto 8192 com embeddings para busca semantica). A comunicacao com o llama-server ocorre via API REST compativel com o formato OpenAI Chat Completions.

Memoria Semantica (semantic-engine.js + LanceDB)

O sistema mantem um banco vetorial local utilizando LanceDB para recuperacao de contexto entre sessoes. Quando o usuario faz uma pergunta, o sistema executa uma busca vetorial nas notas e memorias armazenadas, combinada com busca lexical como fallback. Os resultados sao injetados no prompt do LLM como contexto de memoria, permitindo continuidade entre conversas.

Orquestracao de Chat (chat-service.js)

O fluxo de processamento de uma mensagem segue estas etapas: (1) recebimento da mensagem do usuario via SSE; (2) recuperacao de memoria semantica relevante; (3) descoberta das top-5 skills mais relevantes por similaridade semantica; (4) conversao das skills em definicoes de tools no formato OpenAI; (5) envio do prompt completo para o LLM; (6) decodificacao da resposta entre texto direto ou chamada de tool; (7) execucao da skill e retorno do resultado ao LLM para processamento adicional (ate 3 rodadas no tier Ultra); (8) streaming da resposta final via SSE.

Skill Discovery (keyword-router.js + semantic-engine)

O sistema descobre automaticamente qual skill ativar baseado na intencao do usuario, sem necessidade de configuracao manual. No tier Ultra, utiliza busca semantica no LanceDB sobre nomes, descricoes e tools das skills. Como fallback, utiliza correspondencia lexical por palavras-chave. As top-5 skills sao convertidas em ferramentas disponiveis para o LLM no formato de function-calling.

Persistencia Local (store.js)

Todas as configuracoes, threads de conversa, lembretes e preferencias do usuario sao armazenadas em arquivos JSON no sistema de arquivos local. Nao ha dependencia de banco de dados externo ou servico de nuvem. O sistema nao coleta telemetria nem envia dados para servidores remotos.

Streaming e Comunicacao (websocket.js)

O Node Core mantem um servidor WebSocket para comunicacao em tempo real com o frontend. Os eventos transmitidos incluem: progresso de inicializacao, inicio e fim de sintese de voz, uso de recursos do sistema, lembretes disparados, e traces de observabilidade. O WebSocket tambem faz ponte com o Python Sidecar para eventos de voz.

2.3 Python Sidecar (FastAPI)

O Python Sidecar e um servico FastAPI responsavel exclusivamente pelo processamento de audio. Ele executa em processo separado (porta 8001) e se comunica com o Node Core via HTTP e WebSocket. Suas responsabilidades incluem:

- Deteccao de palavra de ativacao (wake word): utiliza modelos ctranslate2/faster-whisper para detectar as palavras "Luna" ou "Computador" no audio.
- Transcricao de fala (STT): utiliza Whisper para converter audio em texto.
- Sintese de fala (TTS): utiliza Kokoro como mecanismo primario e Edge TTS como fallback, com pre-aquecimento para reducao de latencia.
- Call mode: modo maos-livres com transcricao continua e resposta por voz, sem palavra de ativacao.
- Deteccao de resposta no WhatsApp: identificacao de mensagens de audio recebidas.

O Python Sidecar e opcional: em dispositivos sem microfone ou quando o usuario prefere apenas entrada de texto, ele pode ser desabilitado sem impacto nas demais funcionalidades.

2.4 Comunicacao entre Componentes

Origem	Destino	Protocolo
Frontend (React)	Node Core	HTTP REST + SSE (chat streaming)
Frontend (React)	Node Core	WebSocket (eventos tempo real)
Node Core	Python Sidecar	HTTP REST + WebSocket (voz)
Electron (Main)	Node Core	IPC (process.send)
Node Core	Extension Host	IPC (fork)
Node Core	llama-server	HTTP REST (OpenAI API)

Tabela 1: Comunicacao entre componentes.

3. Ecossistema de Skills e Extensoes

A MomAI e extensivel por design. Skills e extensoes sao modulos que adicionam capacidades ao sistema, seguindo o principio fundamental de que o LLM nunca se comunica diretamente com hardware ou servicos externos -- toda interacao passa por modulos intermediarios.

3.1 Skills Core

As skills core sao modulos embutidos no sistema, registrados automaticamente na inicializacao. Cada skill possui SKILL.md (descricao semantica) e runtime.js (tools expostas ao LLM):

Skill	Funcao	Tools Expostas
Search	Busca na web	search_web(query)
Weather	Clima e previsao	get_weather(location)
Memory	Notas e lembretes	save_note(), recall()
Scheduler	Agendamento	schedule(), list_tasks()
Launcher	Apps e arquivos	launch_app(), open_file()

Tabela 2: Skills core do sistema.

3.2 Extensoes da Comunidade

Extensoes sao modulos instalaveis via loja de extensoes, registrados em community-extensions.json. Cada extensao e um pacote contendo SKILL.md (manifesto), runtime.js (logica), e opcionalmente manifest.json (permissoes) e background-worker.js (processos persistentes).

Extensoes sao executadas em processos isolados via ExtensionHostManager, que utiliza fork() do Node.js para criar ambientes de execucao separados. A comunicacao com o Node Core ocorre via IPC. Extensoes persistentes (ex.: modulo WhatsApp) sao workers continuos que mantem conexoes de longa duracao.

3.3 Skill Discovery

O processo de descoberta ocorre em cada interacao com o LLM. No tier Ultra, o sistema executa busca semantica no LanceDB utilizando embeddings dos nomes, descricoes e ferramentas das skills, combinada com busca lexical como fallback. As cinco skills mais relevantes sao convertidas em tools no formato function-calling do OpenAI e includas no prompt. O LLM decide autonomamente qual tool chamar.

4. Pipeline de Voz

A MomAI possui um pipeline completo de processamento de voz para operacao maos-livres, gerenciado pelo Python Sidecar e orquestrado pelo Node Core.

O fluxo de voz segue estas etapas: (1) WakeWordDetector captura audio do microfone e detecta as palavras "Luna" ou "Computador"; (2) o audio e enviado ao modulo STT (Whisper) para transcricao; (3) o texto transcrito e enviado ao Node Core via HTTP POST para o endpoint /chat/voice-command; (4) o Node Core processa o comando pelo mesmo fluxo do chat, com prefixo de transcricao; (5) a resposta e transmitida via WebSocket para o frontend e simultaneamente enviada ao TTS para sintese de fala.

O sistema oferece dois modos de operacao por voz: (a) modo padrao, onde o usuario diz a palavra de ativacao seguida do comando; (b) call mode, onde o microfone permanece aberto continuamente sem necessidade de palavra de ativacao. O TTS e pre-aquecido na inicializacao (prewarm) para reduzir latencia, utilizando Kokoro como mecanismo primario e Edge TTS como fallback.

5. MomAI Events: Nova Arquitetura

Enquanto a arquitetura atual resolve o problema do assistente conversacional, o MomAI Events propoe uma evolucao para um sistema cognitivo autonomo e orientado a eventos. Inspirado pelo Jarvis (Universo Marvel) como interface que entende linguagem natural e controla tudo, e pela Abelha Rainha (Resident Evil, 1996) como nucleo de inteligencia autonoma que monitora, decide e age, esta nova arquitetura transforma a MomAI de um sistema reativo para um sistema proativo.

5.1 Tres Pilares Fundamentais

Zero Hardcode. O nucleo do sistema nao contem conhecimento previo sobre dispositivos, protocolos, modelos de ML ou comportamentos. Nenhum dispositivo, sensor ou rotina esta fixado no codigo. Tudo e descoberto dinamicamente atraves de manifestos publicados pelos modulos. O sistema nao sabe o que esta conectado ate que um modulo se registre.

Event-Driven. Todas as operacoes do sistema giram em torno de eventos. Sensores publicam mudancas de estado. Timers disparam apos periodo configurado. O LLM so e invocado quando algo relevante acontece. Nao ha polling, checagem constante ou ciclo ocioso. Entre ciclos cognitivos, o sistema consome recursos minimos -- o LLM permanece descarregado da memoria.

Auto-Modificacao. Quando um modulo nao atende uma necessidade identificada pelo LLM, o sistema pode ler, modificar e estender seu codigo para adicionar a funcionalidade faltante. Toda modificacao e testada em sandbox antes de producao. O sistema mantem versionamento para rollback, e o LLM tem limite de tentativas para evitar loop infinito.

5.2 Arquitetura em Tres Camadas

O sistema e dividido em tres camadas com responsabilidades claramente definidas:

Camada	Funcao	Responsabilidades
Edge (Percepcao)	Interface com hardware	Publicar eventos de sensores; executar comandos de atuadores; configuracao
Hub (Orquestrador)	Estado e comunicacao	Blackboard de estados; timers e triggers; payload para LLM; despacho d
LLM (Cognicao)	Decisao e adaptacao	Analisar contexto; decidir acoes e agendamentos; escrever codigo; valid

Tabela 3: As tres camadas da arquitetura MomAI Events.

5.3 Modulos e Manifestos

Cada modulo intermediario se conecta ao Hub e publica um manifesto contendo: nome e identificador unico; categoria (sensor, actuator, perception_module, ou hybrid); descricao em linguagem natural; lista de capacidades; esquema dos dados produzidos (sensores); lista de comandos com parametros tipados (atuadores); linguagem e dependencias; estrutura de arquivos; e ambiente de execucao necessario. O manifesto tambem declara como o modulo aceita ser configurado, informando onde estao seus arquivos-fonte, quais bibliotecas utiliza e como estender suas capacidades.

Modulo	Hardware	Tools e Configuracoes
Visao (Camera)	Cameras USB/IP	detectar_pessoas(), detectar_objetos(), reconhecer_faces()
Audio (Microfone)	Microfones	transcrever(), detectar_palavra_chave(), falar(texto)
Iluminacao	Lampadas Smart	ligar(), desligar(), ajustar_brilho(n), ajustar_cor(c)
Climatizacao	Ar-condicionado	ligar(), desligar(), set_temperatura(g), set_modos(m)
TV / Display	Televisao, Monitor	ligar(), desligar(), exibir(texto), mudar_canal(n)
Travas / Seguranca	Fechaduras	trancar(), destrancar(), get_status()

Tabela 4: Exemplos de modulos e suas capacidades.

5.4 O Ciclo Cognitivo em 6 Etapas

O ciclo cognitivo e o processo central do MomAI Events, executado toda vez que o LLM e invocado para tomar uma decisao:

Etapa 1: Algo Acontece

Um modulo Edge publica um evento no barramento do Hub. O evento pode ser: mudanca detectada por sensor, timer que estourou, comando de voz do usuario, ou trigger composto configurado em ciclo anterior.

Etapa 2: O Hub Monta o Contexto

O Orquestrador interrompe timers pendentes e monta o payload: estado atual de todos os sensores, manifestos dos modulos, politicas ativas do usuario, e registro de triggers configurados em ciclos anteriores.

Etapa 3: O LLM Processa

O modelo analisa o cenário completo, correlaciona o evento com as políticas ativas e decide: ações imediatas, auto-agendamento, ou reconfiguração de módulos.

Etapa 4: O LLM Responde

Resposta estruturada em três blocos: (a) ações -- comandos para módulos; (b) auto-agendamento -- tempo de espera e gatilhos de interrupção; (c) reconfigurações -- alterações nos módulos, incluindo edição de código.

Etapa 5: O Hub Executa

Despacha comandos para os módulos corretos. Se houver reconfigurações, gerencia edição -> teste em sandbox -> aplicação ou rollback.

Etapa 6: O Sistema Espera

Timer criado com o tempo definido pelo LLM. Gatilhos de interrupção podem reiniciar o ciclo antes do tempo. Se o timer estourar, o sistema acorda sozinho para reavaliação. Consumo mínimo durante espera.

Entre ciclos cognitivos, o sistema consome recursos mínimos. Sem polling. Sem checagem constante. Apenas eventos.

5.5 Políticas Ativas

Políticas são instruções em linguagem natural que o usuário fornece ao sistema para definir comportamentos esperados. São armazenadas em tabela persistente e incluídas em todo ciclo cognitivo. O LLM decide quais se aplicam ao contexto atual.

Exemplos: "So liga o ar quando tiver duas ou mais pessoas no ambiente"; "Se eu disser bom dia, me conta o relatório do dia e liga a TV no canal de notícias"; "Se a geladeira ficar aberta por mais de 2 minutos, me avisa no som". O usuário pode adicionar, modificar ou remover políticas a qualquer momento via voz ou texto, sem reinicialização.

5.6 Triggers e Auto-Agendamento

O sistema opera com dois tipos de gatilho. Triggers temporais são definidos pelo usuário ou pelo próprio LLM (ex.: "todo dia às 7h", "daqui a 5 minutos"). O Hub gerencia uma fila de timers. Quando um timer estoura, um evento é publicado.

Triggers de evento são condições que o LLM configura nos módulos. Por exemplo, configurar o módulo de visão para publicar um evento apenas quando exatamente 2 pessoas estiverem no frame. Triggers podem ser simples (um sensor, um valor) ou compostos (AND, OR, NOT entre múltiplos sensores). Toda configuração de trigger é persistida para sobreviver a reinicializações.

5.7 Auto-Modificação de Módulos

Quando um módulo não atende uma necessidade, o sistema pode modificar seu código em seis fases:

- Fase 1 -- Análise: LLM lê o manifesto e o código-fonte do módulo.
- Fase 2 -- Planejamento: define alterações, dependências e novas tools.
- Fase 3 -- Sandbox: cria cópia isolada, implementa, instala dependências.
- Fase 4 -- Validação: executa testes para verificar funcionamento e regressões.
- Fase 5 -- Aplicação: se testes passam, módulo atualizado vai para produção.

- Fase 6 -- Rollback: se falha, tenta corrigir. Apos limite, aborta e avisa.

Mecanismos de segurança: (a) toda modificação e testada em sandbox; (b) versionamento para rollback manual; (c) limite de tentativas para evitar loop infinito; (d) dependências verificadas antes da instalação; (e) LLM avisa se recursos são insuficientes.

6. Integração Híbrida: Request-Response + Event-Driven

O sistema não precisa escolher entre os paradigmas request-response e event-driven. O Hub opera em modo híbrido, com duas portas de entrada que compartilham o mesmo núcleo de processamento.

O Chat Path (request-response) mantém o fluxo atual: usuário digita ou fala, sistema monta contexto, LLM processa e responde, skills são chamadas se necessário, resposta em streaming SSE. É o ciclo familiar de assistente conversacional.

O Event Path (event-driven) segue o ciclo cognitivo completo: sensor publica evento, trigger avalia relevância, payload é montado, LLM decide ações e auto-agendamento, timer criado com gatilhos de interrupção.

Ambos compartilham o mesmo LLM, skills, módulos, políticas, memória e persistência. A diferença é quem inicia o ciclo: o usuário ou um evento autônomo.

O Node Core atual já possui aproximadamente 90% da infraestrutura para se tornar o Hub. Componentes a adicionar: (a) barramento de eventos interno; (b) sistema de triggers; (c) políticas como dados persistentes; (d) timers com suporte a interrupção. Nenhum requer reescrita do núcleo existente.

7. Roteiro de Desenvolvimento

O desenvolvimento segue um plano incremental, partindo da base atual funcional em direção à arquitetura completa do MomAI Events.

Período	Marcos	Status
Hoje	Desktop funcional: chat, voz, skills core, extensões, memória semântica, LLM local com GPU	Concluído
Q3 2026	MomAI Events: barramento de eventos, triggers, políticas ativas, ciclo cognitivo básico	Em andamento
Q4 2026	Computer Use: visão computacional para entender tela e controlar aplicativos	Planejado
Q1 2027	Auto-Modificação: sandbox, versionamento, rollback automático	Planejado
Q2 2027	Marketplace de Extensões: loja para criadores terceiros	Planejado
H2 2027	Mobile + Enterprise: app iOS/Android, gerenciamento centralizado	Planejado

Tabela 5: Roteiro de desenvolvimento.

Cada marco é funcional por si só, e nenhum depende de marcos posteriores para ser útil. O sistema atual já entrega valor como assistente conversacional local; o MomAI Events adiciona capacidades autônomas sem quebrar a experiência existente.

8. Conclusao

A MomAI propoe uma arquitetura que concilia dois paradigmas aparentemente conflitantes: a simplicidade do assistente conversacional request-response e a potencia do sistema cognitivo orientado a eventos. A chave e que ambos resolvem para a mesma operacao fundamental -- montar contexto, consultar o LLM, executar decisoes -- diferenciando-se apenas em quem inicia o ciclo.

O sistema ja implementado demonstra a viabilidade tecnica: um assistente local que combina inferencia de LLM com acoes reais no computador, tudo executado exclusivamente na maquina do usuario. A arquitetura atual processa chat, voz, memorias e skills em fluxo integrado, com streaming em tempo real e extensoes de terceiros em ambiente isolado.

A nova arquitetura MomAI Events estende esse nucleo com capacidades autonomicas: percepcao por sensores, raciocinio ciclico com auto-agendamento, e adaptacao por auto-modificacao de codigo. A integracao hibrida permite que o sistema opere como assistente reativo quando o usuario inicia a interacao, e como agente proativo quando as condicoes do ambiente exigem acao autonoma.

O mercado de IA local esta em aceleracao: Apple Intelligence, Microsoft Copilot e Google Gemini representam o reconhecimento de que o futuro da IA e local. A MomAI se posiciona como a alternativa aberta, com privacidade por construcao e um ecossistema extensivel que qualquer desenvolvedor pode contribuir.

github.com/WesleyQDev | MomAI | Open Source | MIT License